

App Features Technical Specification

This document provides an A-Z detailed technical specification of all the App Features, sequentially numbered from A01 to A43. Features are categorized strictly by their logical domains to guide the Mobile App Frontend Development team.

1. Dashboard & Core Metrics

A01: Welcome Message & Date

- **Data Source:** `UserModel`
- **Implementation:** The app fetches the currently logged-in user's `name` from the `UserModel` and uses device time to determine the greeting (e.g., "Good Morning, {name}").

A02: Member & Task Statistics

- **Data Source:** `UserModel` , `LeaveApplicationModel` , `TaskModel`
- **Implementation:**
 - *Total Active/Alumni:* Query `UserModel` where `status` is 'active' or 'alumni'.
 - *Members on Leave:* Query `LeaveApplicationModel` where `status` = 'approved' and the current date falls within the `duration` object.
 - *Active Tasks:* Query `TaskModel` for tasks assigned to the user where `status` is 'in_progress'.

A03: Today's Active Meeting

- **Data Source:** `MeetingModel`
- **Implementation:** Query `MeetingModel` where `date_time` matches today's date. The card displays `title` , `date_time` , and `agenda` . Clicking it navigates to the Meeting Details screen, passing the `_id` .

A04: My Tasks

- **Data Source:** `TaskModel`
- **Implementation:** Query `TaskModel` where the user's `_id` is inside the `assigned_to` array. Displays `name` and domain (via `domain_id`).

A05: Quick Action Buttons

- **Implementation:** UI-level shortcuts (e.g., "Apply Leave", "New Task") that navigate to respective screens. Visibility controlled by `RoleModel` policies.

A06: Community Updates & Task Load Graph

- **Data Source:** `UpdatesModel` , `TaskModel`
- **Implementation:**
 - *Updates:* Query `UpdatesModel` for the latest broadcasts (using `title` and `content`).
 - *Task Load:* Aggregation query on `TaskModel` grouping by `status` and `domain_id` to render the visual graph.

A07: Global Search Bar (Universal Search)

- **Data Source:** `UserModel` , `TaskModel` , `MeetingModel` , `RepositoryModel`
- **Implementation:** A universal search bar executing text searches across tasks, meetings, members, and documents for quick navigation within the App.

2. Tasks & Calendar

A08: Calendar View

- **Data Source:** `TaskModel` , `MeetingModel`
- **Implementation:** Fetches all tasks and meetings for the month. Uses `deadline` (Tasks) and `date_time` (Meetings) to visually pin them on the calendar.

A09: Tasks List View

- **Data Source:** `TaskModel` , `NotificationModel`
- **Implementation:** Lists tasks sorted by `deadline` . Includes the reassignment feature which triggers a new `NotificationModel` document for assigned/unassigned members.

A10: Add Task Button (Admin/Authority)

- **Data Source:** `TaskModel` , `RoleModel` , `NotificationModel`
- **Implementation:** Checks IAM RBAC. Creates a `TaskModel` document with `name` , `deadline` , `mentor_ids` , `description` , `type` , `priority` , `attachments` , `assigned_to` , and `domain_id` .

A11: Task Filters (Completed, In Progress, Not Started)

- **Data Source:** `TaskModel`
- **Implementation:** UI toggle that queries the tasks list strictly by the `status` field in `TaskModel` .

A12: Task Workflow & Table

- **Data Source:** `TaskModel`
- **Implementation:** Users accept tasks (updates `status` to 'in_progress'). Submission appends an object to `TaskModel.submissions` . Mentors can review via `TaskModel.reviews` array.

A13: Task Activity Log (Audit Trail)

- **Data Source:** `TaskModel`
- **Implementation:** A timeline in Task Details rendering the `TaskModel.history` array to track assignments and status changes.

3. Meetings

A14: Meeting Dashboards

- **Data Source:** `MeetingModel`

- **Implementation:** Fetches and displays all meetings as informational cards.

A15: Create Meeting

- **Data Source:** `MeetingModel` , `RoleModel`
- **Implementation:** Validates admin authority via RBAC. Creates a `MeetingModel` document populated with `title` , `agenda` , `venue` , `target_batch` , and `target_domain` .

A16: Meeting Filters

- **Data Source:** `MeetingModel`
- **Implementation:** UI filters to categorize meetings by `status` ('upcoming', 'ongoing', 'completed').

A17: Meeting Summary & Attendance

- **Data Source:** `MeetingModel` , `AttendanceModel`
- **Implementation:** Admins update `MeetingModel.mom_content` . Attendance tracking creates/updates `AttendanceModel` documents linking `meeting_id` and `user_id` .

A18: Integrated Meeting

- **Implementation:** Uses `MeetingModel.venue.link` to deep-link to Google Meet or other platforms from the App.

4. Registry / Analytics

A19: Tables of Works

- **Data Source:** `TaskModel`
- **Implementation:** Queries `TaskModel` where `status` = 'completed'. Displays who completed it. Export feature parses this JSON to CSV.

A20: Tables of Meeting

- **Data Source:** `AttendanceModel` , `MeetingModel`

- **Implementation:** Aggregates `AttendanceModel` for a specific `meeting_id` to count present vs. absent members.

5. Leave Application

A21: Leave Application Management

- **Data Source:** `LeaveApplicationModel`
- **Implementation:** User creates a record with `user_id`, `leave_type`, `target`, `duration`, and `reason`. Defaults to `status` = 'pending'.

6. Repository

A22: Essential Documents

- **Data Source:** `RepositoryModel`
- **Implementation:** Queries `RepositoryModel` based on user's domain and role. Displays `name`, `type`, and provides links from `file_urls`.

7. Forms

A23: Active Form Cards

- **Data Source:** `FormModel`
- **Implementation:** Queries `FormModel` where `deadline` > current time.

A24: Form Fillup Fields

- **Data Source:** `FormModel`, `FormSubmissionModel`
- **Implementation:** Dynamically renders UI based on `fields` array in `FormModel`. On submit, creates a `FormSubmissionModel` linking `form_id` and `user_id`.

A25: Form Instructions

- **Data Source:** `FormModel`
- **Implementation:** Displays instructions dynamically read from the `FormModel`.

A26: Form Responses Dashboard (Admin Feature)

- **Data Source:** `FormSubmissionModel`, `FormModel`
- **Implementation:** Admin can view all submissions for a specific form. Renders the JSON `responses` in a table or graphical format within the App, with an option to export to CSV.

8. Profile, Settings & Auth

A27: User Profile

- **Data Source:** `UserModel`, `TaskModel`
- **Implementation:** Fetches `profile_pic`, `name`, `domain`, and `task_count` from models.

A28: Sidebar Options

- **Implementation:** Global App UI navigation menu (Drawer or Bottom Tab).

A29: Dark / Light Mode

- **Implementation:** App UI theme toggle, saves preference locally on the device.

A30: Developer Mode

- **Implementation:** Hidden toggle that enables extra debugging tools within the App.

A31: Login & Auth

- **Data Source:** `UserModel`
- **Implementation:** Authenticates via email / password or Google OAuth.

A32: Detailed Documentation

- **Implementation:** Static guide/help section for normal members.

A33: Terms and Conditions

- **Implementation:** Static page displaying the official rules.

A34: In-App Notification Center (Bell Icon)

- **Data Source:** `NotificationModel`
- **Implementation:** A bell icon opening a list of the user's notifications.

A35: Team Directory (For Normal Members)

- **Data Source:** `UserModel`
- **Implementation:** A directory/search screen allowing normal members to view their peers.

A36: Forgot Password & Reset Flow

- **Data Source:** `UserModel`
- **Implementation:** Standard OTP/Email flow for password reset, hashing the new password.

9. Admin Authority

A37: Admin Dashboard

- **Data Source:** Aggregation across Models
- **Implementation:** Accessible only if `RoleModel` grants admin policies. Shows high-level metrics.

A38: Member Management (List of all members)

- **Data Source:** `UserModel` , `RoleModel`
- **Implementation:** Admins can modify `status` (make alumni) and update

`volatile_data` in the `UserModel` .

A39: Application Approvals

- **Data Source:** `LeaveApplicationModel`
- **Implementation:** Admins fetch 'pending' leaves. On action, updates `status` and `reviewed_by` .

A40: Dynamic Roles & Permissions Manager (IAM UI)

- **Data Source:** `RoleModel` , `PolicyModel`
- **Implementation:** Dedicated Superadmin screen in the App to create custom roles, assign specific access policies, and link them to users dynamically.

A41: Bulk Member Upload (CSV)

- **Data Source:** `UserModel`
- **Implementation:** Allows Admins to upload a CSV of member details for batch insertion directly from the App.

10. Chat Feature

A42: Core Chat

- **Data Source:** `ChannelModel` , `ChatModel`
- **Implementation:** Group chat using `ChannelModel` and `ChatModel` with WebSockets.

A43: Advanced Chat (File Sharing & Mentions)

- **Data Source:** `ChatModel` , `RepositoryModel` , `NotificationModel`
- **Implementation:** Enhances the chat by allowing file attachments and `@mentions` .